

Capítulo 1

Do simulador ao carro real

Como identificar a planta de verdade a partir dos logs que você já tem e reprojeter o controle de velocidade, sem adivinhar nada.

Você projetou um controlador de velocidade no simulador. Ele foi identificado com cuidado, validado com previsão cega, e funcionava. Copiou para o carro real, rodou, e o carro disparou: com referência de 1 m/s, ficou andando a 1,85 m/s durante o ensaio inteiro, e o controlador não fez nada para corrigir.

Este capítulo é a investigação desse problema, do começo ao fim, com o raciocínio explícito em cada passo. Não há nada aqui que exija genialidade: há um método. Cada passo responde a uma pergunta, e cada resposta vem de uma fonte que você consegue conferir — o código, a física ou os dados.

O que você vai aprender

1. Ler o código antes dos dados, para descobrir a *estrutura* da planta.
2. Modelar só o que o código não diz, com poucos parâmetros e significado físico.
3. Fazer inventário e triagem dos logs, descartando com evidência.
4. Reconstruir sinais que não foram medidos.
5. Ajustar o modelo pela trajetória inteira (erro de saída).
6. Validar: testes físicos, previsão cega e o que os dados *não* conseguem dizer.
7. Descobrir, pelos próprios logs, que código estava rodando.
8. Montar um simulador do carro real e validá-lo contra os ensaios.
9. Projeter o controlador: estrutura, ganho nominal e ganho proporcional.

1.1 O sintoma

O controlador que rodou no carro é o do simulador: um PI com *feedforward* da referência, compensação do atrito identificado ($a_0 = 0,096 \text{ m/s}^2$, $c = 0,00425 \text{ 1/m}$), um filtro passa-baixas no comando e saturação $u \in [0, 1]$. No simulador, a planta identificada era

$$\dot{v} = Ku - cv|v| - a_0 \text{sign}(v), \quad K \approx 1,0, \quad (1.1)$$

ou seja, u era uma *aceleração*: com $u = 0$, o atrito freava o carro.

Nos ensaios do carro real (24/09, carro roxo), o comportamento foi outro:

referência	velocidade em regime	quanto tempo ficou lá
1,0 m/s	1,74 a 1,87 m/s	até o fim do ensaio
0,5 m/s	1,31 a 1,41 m/s	até o fim do ensaio

E durante todo esse tempo o comando ficou em $u = 0$.

Pare e pense

Se u fosse aceleração, como na equação (1.1), o que deveria acontecer com um carro a 1,85 m/s e $u = 0$ durante 17 s? E o que os logs mostram? A diferença entre as duas respostas é a pista de que a planta do carro real não é a do simulador.

1.2 Passo 1: leia o código antes dos dados

A primeira tentação é abrir os logs e ajustar a equação (1.1) de novo. Resista. Antes de olhar dado, descubra *o que o sinal u faz* até chegar à roda. Essa informação está no código, e o código não tem ruído.

1.2.1 O caminho do comando

O controlador chama `car.set_u(u)`. Seguindo a chamada:

veiculo_real/fva_car/car.py

```
def set_u(self, u):
    self.u = np.clip(u, -CAR['ACCELMAX'], CAR['ACCELMAX']) # u em [-1, 1]
    if np.abs(self.v) > CAR['VELMAX']:
        self.u = 0.0
    F = CAR['MASS']*self.u # "controlador linearizante"
    T = CAR['RW']*np.sum(F) # pseudo-torque
    self.atuador.set_torque(T)
```

veiculo_real/fva_car/servos.py

```
GAIN_TORQUE_FORWARD = 0.4 # [rad/s] por unidade de pseudo-torque

def set_torque(self, T):
    self.dth_pwm = self.gain_torque * T # T vira uma TAXA de PWM

# thread "actuator", a cada self.dt = 0.03 s:
if self.gear == Gear.FORWARD:
    self.th_pwm += self.dth_pwm * self.dt # integra
    self.th_pwm = np.clip(self.th_pwm, 0.0, self.max_throttle) # 0 a 18,1 graus
```

A linha decisiva é `self.th_pwm += self.dth_pwm * self.dt`. O comando não vira um torque nem uma aceleração: vira a *taxa de variação* do PWM do acelerador, e uma thread separada integra essa taxa. Juntando as constantes ($m = 5,16$ kg, $R_W = 0,08$ m, ganho 0,4):

$$\dot{\theta} = k_{\theta} u, \quad k_{\theta} = 0,4 \cdot 0,08 \cdot 5,16 = 0,165 \text{ rad/s} = 9,46^\circ/\text{s por unidade de } u, \quad (1.2)$$

com θ (o PWM acima do neutro) limitado a $[0^\circ, 18,1^\circ]$.

Ideia-chave

O nome de uma variável não diz o que ela faz; o código diz. No simulador, u era aceleração. No carro real, u é a *velocidade com que o acelerador anda*. Isso coloca um integrador puro

entre o controlador e o motor, antes de qualquer física do carro. Com $u = 0$, o acelerador fica parado onde está — e o carro mantém a velocidade.

É possível ter u negativo no carro real?

Sim. O `set_u` aceita $u \in [-1, 1]$, o `set_torque` repassa o sinal, e a thread subtrai do PWM. O que $u < 0$ faz fisicamente é o que importa:

- **baixa o acelerador**, até o neutro ($\theta = 0$) e não além: em marcha à frente a thread limita $\theta \geq 0$. Não é freio nem ré — é tirar o pé do acelerador e deixar o carro ir no embalo. A ré exige `set_reverse()`, um procedimento de 5 s que o `set_u` nunca chama;
- **o próprio código do professor usa $u < 0$** : o PD do `set_vel` dá $u < 0$ sempre que $v > v_{\text{ref}}$, e o `stop_mission` aplica $u = -1$ para parar;
- **os logs confirmam** (Seção 1.7.2): no ensaio 003552, a parada do ultrassom levou u a $-0,68$, o PWM reconstruído caiu de $18,1^\circ$ para $9,3^\circ$ e o carro desacelerou de $1,5$ para $0,9$ m/s.

O limite físico: para desacelerar, o carro depende do atrito (embalo), então desacelera mais devagar do que acelera. Isso entra no projeto como um caso de robustez.

1.2.2 O caminho da medição

A velocidade que o controlador enxerga também tem uma história:

- o **encoder** (firmware `odometer.ino`) conta pulsos em janelas de 30 ms e manda um valor por janela, com sinal (quadratura);
- o `car.py` passa esse valor por uma **média móvel de 30 amostras** (`MovingAverage(n=30)`), que chamaremos de MA30:

$$v_{\text{med}}[k] = \frac{1}{N} \sum_{i=0}^{N-1} v_{\text{enc}}[k-i], \quad N = 30. \quad (1.3)$$

Uma média de N amostras espaçadas de T tem fase linear na frequência: ela atrasa o sinal de

$$\text{atraso de grupo} = \frac{N-1}{2} T. \quad (1.4)$$

Esse atraso depende de T , o período do laço. E o período não é o que o código promete.

1.2.3 O relógio do laço

O `step()` dorme até completar 20 ms desde a última leitura e só *depois* lê os sensores (encoder, IMU), o que leva cerca de 9 ms. O período real é a soma. A Figura 1.1 mostra a distribuição medida nos 15 logs: mediana de 28,8 ms, com 5% das amostras acima de 46 ms. Com $T = 28,8$ ms, a equação (1.4) dá 0,42 s de atraso só na média móvel — mais de oito vezes o atraso de 0,05 s que o projeto do simulador assumia.

A Figura 1.2 mostra a ordem das operações em cada iteração. Ela é importante para interpretar o log: a linha k é gravada *antes* de o controle calcular o novo comando, então o u gravado na linha k é o que agiu sobre o carro entre t_{k-1} e t_k .

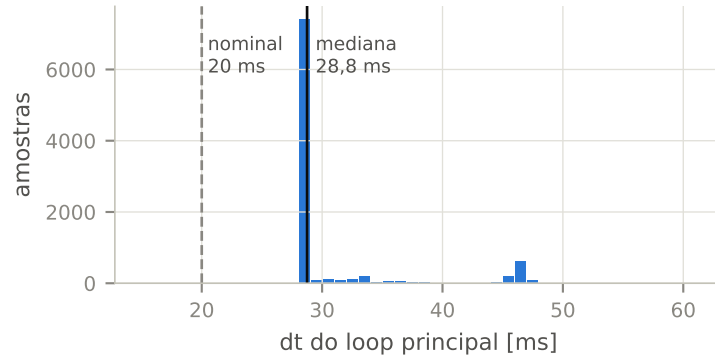


Figura 1.1: Período do laço principal nos 15 ensaios. O código pede 20 ms; a leitura dos sensores vem depois da espera e soma cerca de 9 ms.

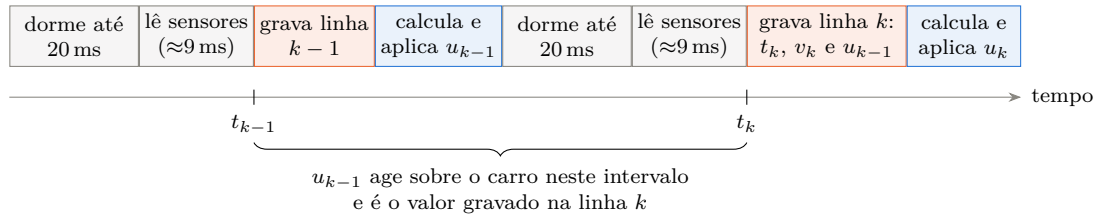


Figura 1.2: Duas iterações do laço principal. O instante t_k é carimbado ao fim da leitura dos sensores.

1.3 Passo 2: modele só o que o código não diz

O código determinou o começo e o fim da cadeia: o integrador do PWM e a média móvel. O que fica no meio — como o PWM vira velocidade — depende do ESC, do motor, da bateria e do chão, e isso o código não sabe. É aqui, e só aqui, que entram parâmetros a identificar.

A escolha da estrutura vem da física de um motor CC:

- **ganho estático com zona morta.** Em regime, a velocidade de um motor CC é proporcional à tensão aplicada, e o PWM define essa tensão. Mas o ESC tem uma faixa morta perto do neutro, em que o motor não gira. O próprio `servos.py` traz uma calibração com essa forma (uma reta com *offset*). Então $v_{ss} = G \max(0, \theta - DB)$;
- **atraso de primeira ordem.** A inércia do motor e do carro faz a velocidade chegar ao valor de regime exponencialmente, com constante de tempo mecânica τ ;
- **atraso puro d .** A janela do encoder e a leitura serial.

Modelo do carro real

$$\begin{aligned}
 \dot{\theta} &= k_{\theta} u, & \theta &\in [0^{\circ}, 18,1^{\circ}] & (\text{código}) \\
 v_{ss} &= G \max(0, \theta - DB) & & & (\text{a identificar}) \\
 \tau \dot{v} &= v_{ss} - v & & & (\text{a identificar}) \\
 v_{\text{med}} &= \text{MA30}(v(t-d)) & & & (\text{código} + \text{a identificar})
 \end{aligned} \tag{1.5}$$

Quatro parâmetros livres: G , DB , τ e d .

A Figura 1.3 junta tudo. Em azul, o que veio do código; em laranja, o que precisa sair dos dados.

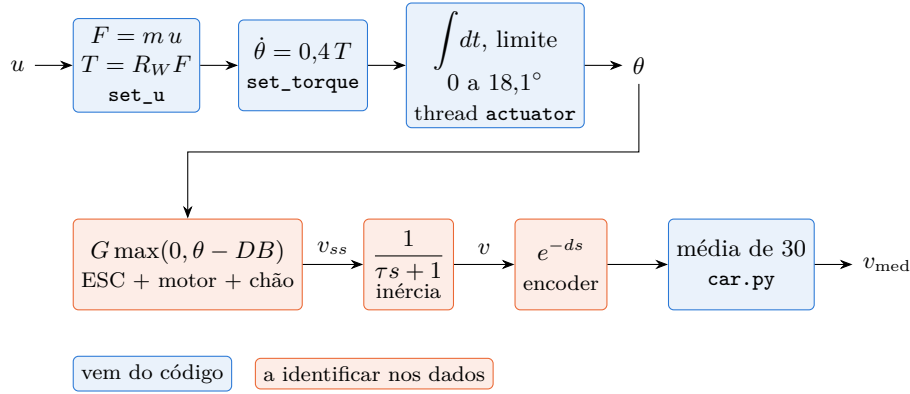


Figura 1.3: A cadeia do comando até a velocidade medida no carro real.

Armadilha

Não escolha a estrutura pelo ajuste. Um modelo com mais termos sempre ajusta melhor, mas só a estrutura certa *prevê*. Aqui a estrutura veio do código (integrador, média móvel) e da física (ganho, zona morta, inércia); o ajuste só dá os números.

1.4 Passo 3: inventário e triagem dos dados

Havia quatro pastas de logs. A primeira verificação foi um *hash* (md5) de cada arquivo: as pastas eram cópias cumulativas umas das outras, com arquivos idênticos. No total, 15 ensaios distintos, todos de 20 s.

Nem todo ensaio tem informação sobre a planta. Um ensaio só serve para identificar se o carro de fato respondeu ao comando. Para decidir isso sem achismo, há uma testemunha independente do encoder: a IMU. Um carro parado tem aceleração medida quase constante; um carro andando ou sendo carregado na mão, não (Figura 1.4).

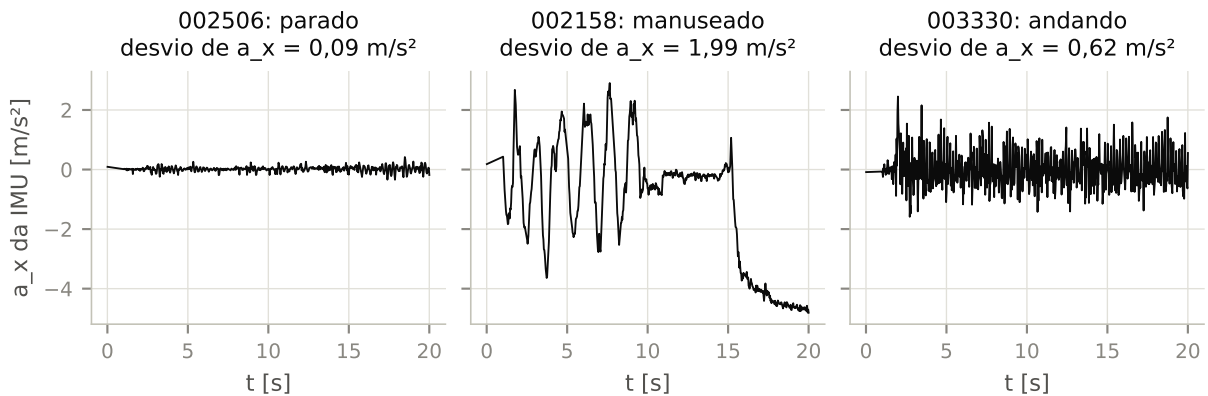


Figura 1.4: A IMU como testemunha. No 002506, $u = 1$ por 18,7 s e a IMU não se mexe: o motor não respondeu (bateria fraca). No 002158, o encoder marca zero, mas a IMU está agitada: o carro estava sendo manuseado. No 003330, o carro andou.

A Tabela 1.1 mostra o resultado. Os ensaios destacados (sete) são os que servem para identificar; cada descarte tem uma evidência na própria tabela:

- **000844:** u máximo de $0,30 = 0,4 \times 0,7$, exatamente o PD do `set_vel(0,7)` do professor. Não era o controlador em estudo;
- **002506:** $u = 1$ e a IMU parada: o motor não respondeu (bateria fraca);

Tabela 1.1: Triagem dos 15 ensaios. O desvio padrão de a_x (IMU) separa carro parado ($\approx 0,1 \text{ m/s}^2$) de carro em movimento ou manuseado.

ensaio	hora	$\max v $	u (mín / máx)	v_{ref} máx	desvio a_x	classificação
000844	00:08	0,16	0,00 / 0,30	0,70	0,50	código do professor
002158	00:21	0,00	-0,04 / 1,00	1,00	1,99	manuseado
002236	00:22	0,36	-0,11 / 1,00	1,00	1,24	manuseado
002318	00:23	0,01	-0,03 / 1,00	1,00	0,63	manuseado
002506	00:25	0,00	0,00 / 1,00	1,00	0,09	parado (bateria fraca)
002634	00:26	1,84	0,00 / 1,00	1,00	0,56	bateria boa
002751	00:27	1,87	0,00 / 1,00	1,00	0,50	bateria boa
003019	00:30	0,00	0,00 / 0,00	0,00	0,08	parada do ultrassom
003157	00:31	0,00	0,00 / 0,00	0,00	0,06	parada do ultrassom
003238	00:32	0,00	-0,01 / 1,00	0,95	0,09	parada do ultrassom
003330	00:33	1,87	0,00 / 1,00	1,00	0,62	bateria boa
003552	00:35	1,51	-0,68 / 1,00	1,00	0,89	descarregando
003904	00:39	1,78	0,00 / 1,00	1,00	1,41	descarregando
004516	00:45	1,35	0,00 / 1,00	0,50	1,26	descarregando
004851	00:48	1,41	0,00 / 1,00	0,50	1,41	descarregando

- **002158, 002236, 002318:** IMU agitada sem leitura coerente do encoder: manuseio;
- **003019, 003157, 003238:** $v_{\text{ref}} = 0$ quase o tempo todo: a parada por obstáculo do ultrassom estava ativa.

Ideia-chave

Descartar exige evidência. Um log descartado “porque ficou estranho” é um viés que você colocou no modelo. Um log descartado porque a IMU prova que o carro estava parado com $u = 1$ é informação: ele diz que, naquele momento, o motor não estava sendo alimentado.

1.5 Passo 4: reconstrua o que não foi medido

O PWM do acelerador, θ , não é gravado. Mas o código diz exatamente como ele é calculado (equação (1.2)), e o u que o alimenta está no log. Então dá para refazê-lo, integrando u como a thread faz, com o limite de 0° a $18,1^\circ$ e respeitando a ordem da Figura 1.2:

$$\theta_k = \text{clip}(\theta_{k-1} + k_\theta u_k (t_k - t_{k-1}), 0, 18,1^\circ). \quad (1.6)$$

A Figura 1.5 mostra o resultado para o ensaio 003330. Enquanto $u = 1$, o PWM sobe a $9,46^\circ$ por segundo; quando u cai para zero, o PWM *congela* em $14,3^\circ$ (79% do limite). A velocidade segue o PWM com atraso e fica em $1,85 \text{ m/s}$.

Aqui o sintoma do início do capítulo se explica sozinho: o controlador só podia mandar $u \geq 0$. Quando o carro passou da referência, o melhor que ele conseguia fazer era $u = 0$ — que, neste carro, significa *manter o acelerador*. É uma catraca: o PWM só sobe.

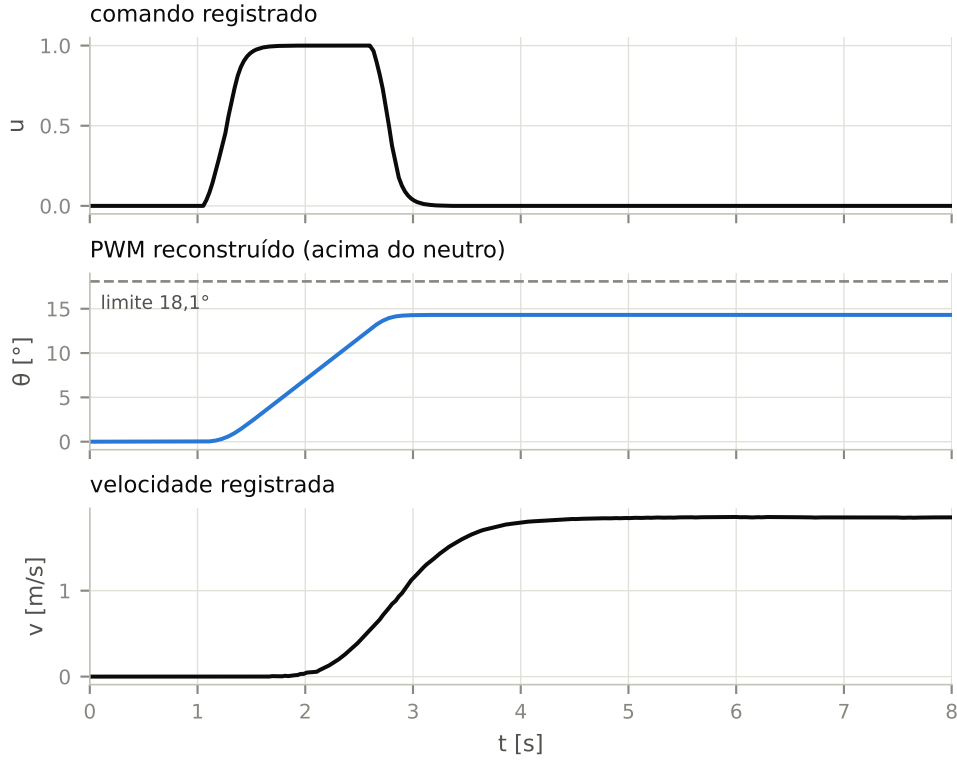


Figura 1.5: Ensaio 003330. De cima para baixo: o comando registrado, o PWM reconstruído pela integração do código e a velocidade registrada. Com $u = 0$, o acelerador fica parado — e o carro também fica na mesma velocidade.

1.6 Passo 5: ajuste pela trajetória inteira

Há duas formas de ajustar um modelo dinâmico a dados:

- **erro de equação:** estimar $\dot{v}$ a partir dos dados e ajustar a equação amostra a amostra, usando a velocidade *medida* em cada passo. É fácil de resolver (mínimos quadrados lineares), mas é um teste fácil de passar: o erro de um passo não se propaga para o próximo;
- **erro de saída:** dar ao modelo *só* o comando registrado, simular o ensaio inteiro e comparar a trajetória simulada com a registrada. O erro se acumula, então o modelo precisa acertar a dinâmica, não só a derivada.

Aqui usamos erro de saída, pelo mesmo motivo do `IDENTIFICACAO.md` (§7): é o critério que corresponde ao uso do modelo — prever o que o carro vai fazer. O custo é

$$J(G, DB, \tau, d) = \sum_k (\hat{v}_k(G, DB, \tau, d) - v_k)^2, \quad (1.7)$$

em que $\hat{v}_k$ é a saída do modelo (1.5) simulado a partir do u registrado, passando pela mesma média de 30 amostras. Como $\hat{v}$ depende dos parâmetros de forma não linear (zona morta, exponencial, limites), o mínimo é encontrado por mínimos quadrados não lineares (função `least_squares` do `scipy`), com limites físicos para cada parâmetro ($G > 0$, $DB \geq 0$, $\tau > 0$, $d \geq 0$).

A Tabela 1.2 e a Figura 1.6 mostram o resultado: o modelo acompanha as trajetórias com erro de 1 a 2,5 cm/s. A última coluna de parâmetros merece atenção: K é o número que o controlador vai precisar, porque é ele que diz “quanta aceleração cada unidade de u produz”:

$$K = G \cdot \frac{180}{\pi} \cdot k_\theta \left[\frac{\text{m/s}}{\text{grau}} \cdot \frac{\text{grau}}{\text{rad}} \cdot \frac{\text{rad/s}}{u} = \frac{\text{m/s}^2}{u} \right]. \quad (1.8)$$

Tabela 1.2: Parâmetros ajustados por ensaio. $K = G \cdot \frac{180}{\pi} \cdot k_\theta$ é o ganho do integrador visto pelo controlador, em m/s^2 por unidade de u .

ensaio	hora	G [m/s/°]	DB [°]	τ [s]	d [s]	K	rmse [m/s]
002634	00:26	0,158	3,03	0,289	0,000	1,49	0,009
002751	00:27	0,169	3,49	0,259	0,000	1,60	0,011
003330	00:33	0,170	3,42	0,251	0,000	1,61	0,009
003552	00:35	0,113	2,31	0,760	0,049	1,07	0,016
003904	00:39	0,112	2,52	0,749	0,000	1,06	0,017
004516	00:45	0,093	1,11	0,634	0,000	0,88	0,014
004851	00:48	0,099	1,76	0,616	0,000	0,93	0,025

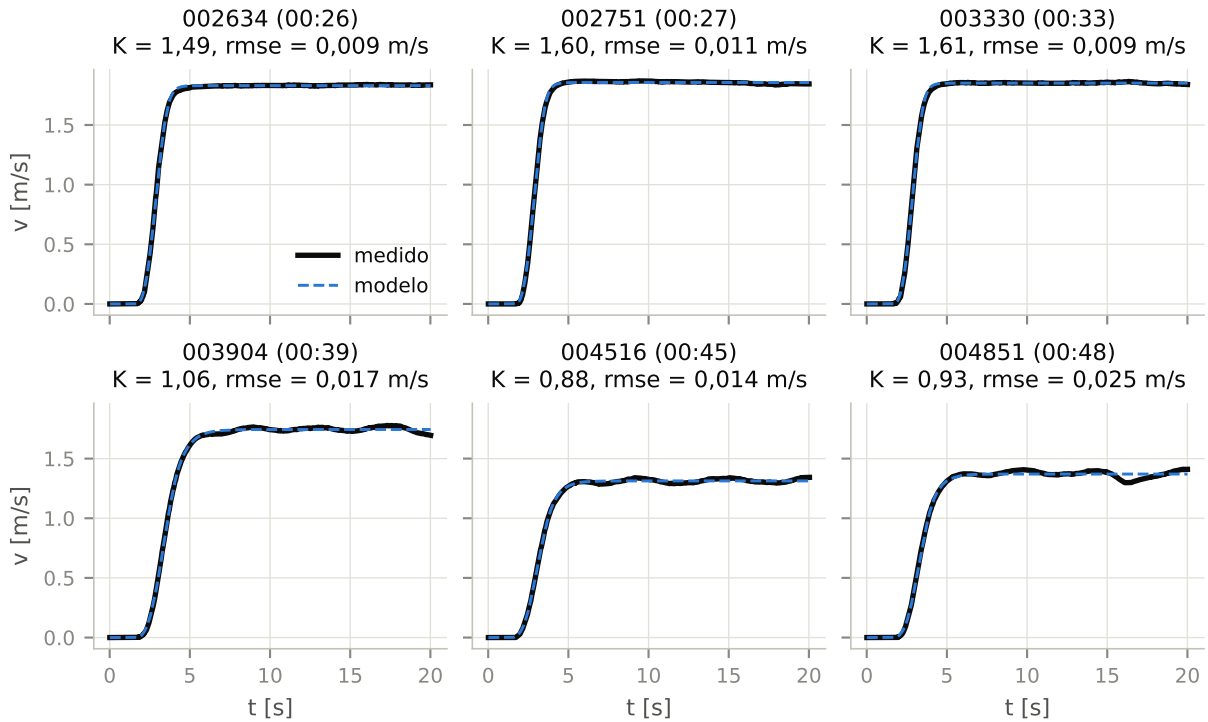


Figura 1.6: Ajuste por erro de saída em seis ensaios: velocidade medida (preto) e modelo simulado só a partir do u registrado (azul tracejado).

1.7 Passo 6: validar é mais importante que ajustar

Um bom ajuste não prova que o modelo está certo. Três perguntas diferentes precisam de resposta: o modelo tem a física certa? Ele prevê o que não viu? E o que os dados *não* conseguem determinar?

1.7.1 Com $u = 0$, a velocidade fica parada

A diferença mais importante entre a planta do simulador (1.1) e a do carro real (1.5) aparece quando $u = 0$. No simulador, o carro desacelera $a_0 + cv^2 \approx 0,11 \text{ m/s}^2$. No modelo real, o PWM congela e a velocidade fica constante. Os logs decidem:

ensaio	velocidade com $u = 0$	$\dot{v}$ medido	$\dot{v}$ pelo modelo do simulador
002634	1,83 m/s	+0,0007	-0,110
002751	1,86 m/s	-0,0020	-0,111
003330	1,85 m/s	-0,0002	-0,111
003904	1,74 m/s	+0,0013	-0,109
004516	1,31 m/s	+0,0011	-0,103
004851	1,37 m/s	-0,0015	-0,104

A Figura 1.7 mostra o mesmo teste de forma visual: o mesmo u aplicado às duas plantas. O modelo do simulador pararia o carro em $t \approx 16$ s; o carro real ficou a 1,85 m/s, e o modelo identificado acompanha.

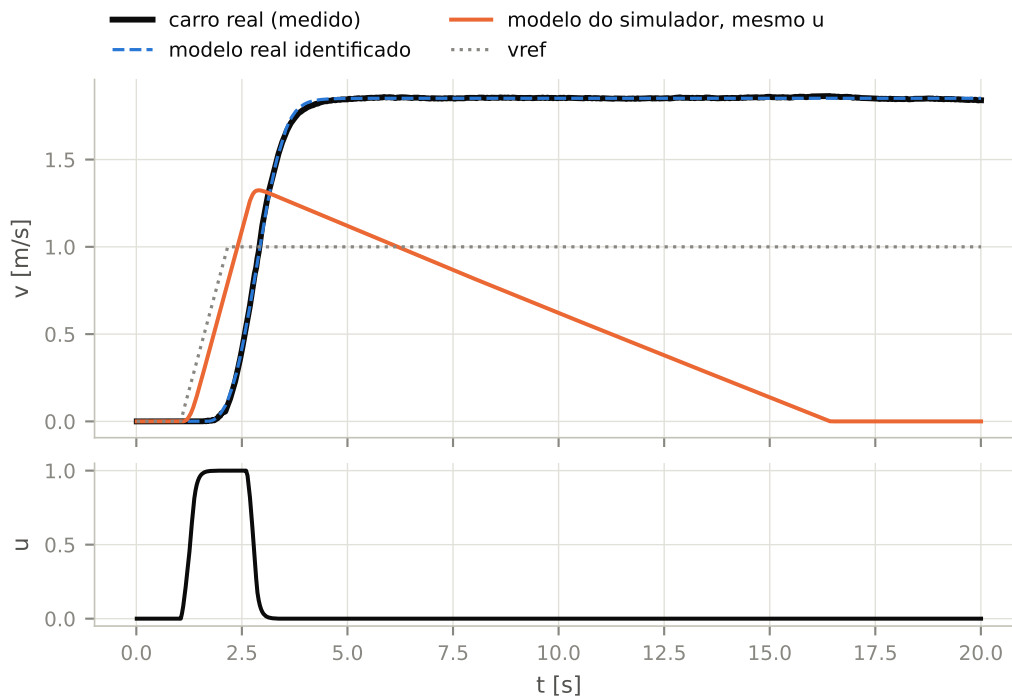


Figura 1.7: Ensaio 003330: o mesmo comando aplicado ao modelo do simulador (laranja) e ao modelo identificado do carro real (azul). Só o segundo explica o que o carro fez.

1.7.2 O acelerador também desce

Em quase todos os ensaios o PWM só subiu, porque o controlador não mandava $u < 0$. Um modelo validado só com o acelerador subindo não prova que o integrador funciona para baixo. O ensaio 003552 é a exceção: a parada do ultrassom chamou o `set_vel(0)`, que mandou u até $-0,68$. A Figura 1.8 mostra que o PWM reconstruído desceu, a velocidade caiu de 1,5 para 0,9 m/s e o mesmo modelo acompanhou a descida e a nova subida (rmse de 0,016 m/s). É a evidência experimental de que $u < 0$ funciona no carro real.

Um detalhe dessa figura voltará no projeto: ajustando constantes de tempo diferentes para subir e para descer, a descida sai cerca de duas vezes mais lenta ($\tau \approx 1,2$ s contra 0,6 s). Faz sentido: para acelerar, o motor empurra; para desacelerar, só o atrito freia.

1.7.3 Previsão cega — e o erro de misturar condições

O teste que realmente vale é prever um ensaio que o modelo não viu. O procedimento *deixa um de fora*: ajusta os parâmetros com todos os ensaios menos um e prevê o que ficou de fora.

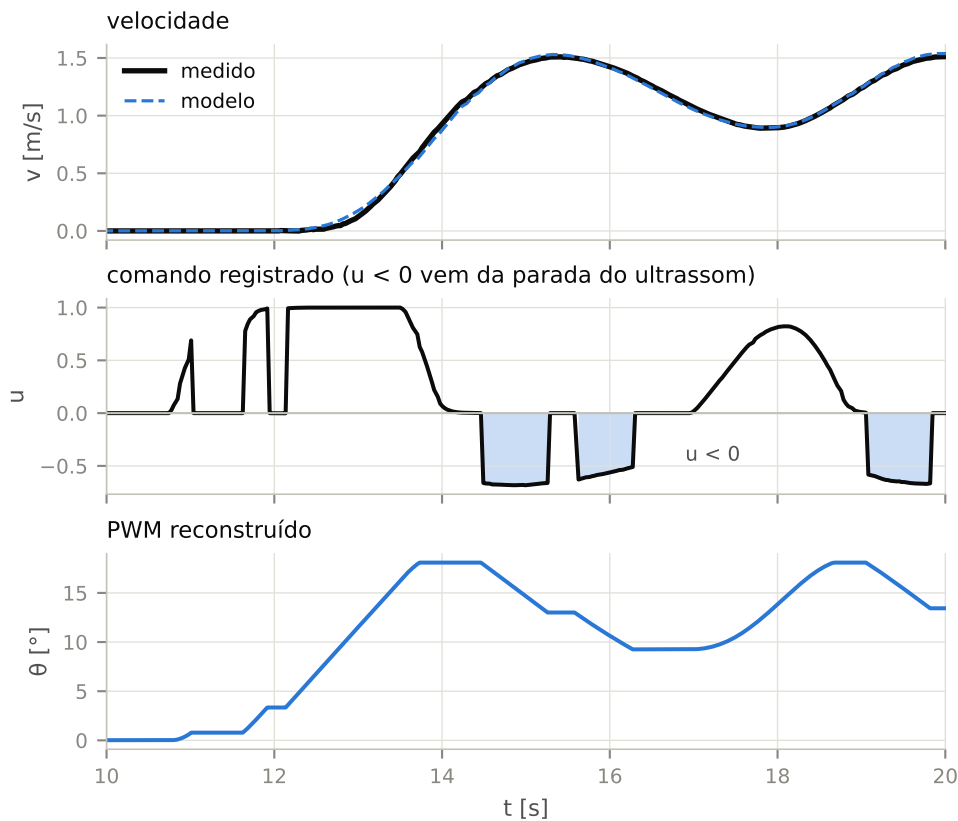


Figura 1.8: Ensaio 003552: $u < 0$ (área azul) baixa o PWM, e o carro desacelera. O modelo (azul tracejado) acompanha a descida e a subida.

A primeira tentativa misturou todos os ensaios e falhou. A Tabela 1.2 mostra por quê: os parâmetros formam dois grupos. Das 00:26 às 00:33, $K \approx 1,5\text{--}1,6$ e $\tau \approx 0,26$ s; a partir das 00:35, K cai para 1,07 e continua caindo até 0,88, enquanto τ sobe para 0,6–0,76 s (Figura 1.9). É a bateria descarregando: tensão menor dá menos velocidade por grau de PWM (menor G), e a resistência interna maior deixa o motor mais lento (maior τ). Um modelo único não pode prever os dois estados.

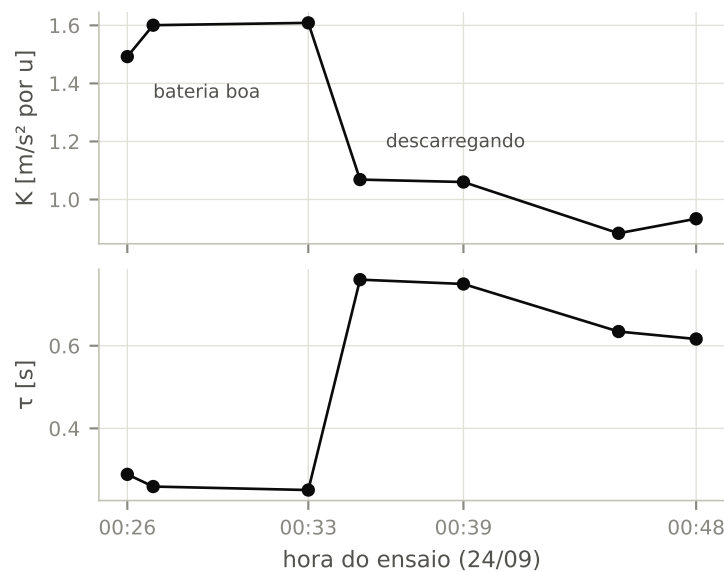


Figura 1.9: Parâmetros ajustados ao longo da sessão. A planta muda com a carga da bateria.

Separando por estado da bateria:

grupo	ensaio previsto	rmse da previsão cega [m/s]
bateria boa	002634 / 002751 / 003330	0,063 / 0,023 / 0,044
descarregando	003552 / 003904 / 004516 / 004851	0,110 / 0,132 / 0,063 / 0,060
misturando os grupos	003552	0,164

Na bateria boa, a previsão erra de 2 a 6 cm/s. No grupo que descarregava, o erro é maior porque o próprio K continuava caindo de um ensaio para o outro.

Armadilha

Nunca misture num ajuste só ensaios de condições diferentes — rampa e pista plana no simulador, bateria cheia e fraca aqui. O ajuste vira uma média que não descreve nenhuma das condições, e a previsão cega falha. Antes de juntar ensaios, verifique se os parâmetros ajustados *separadamente* são parecidos.

1.7.4 O que os dados não conseguem dizer

Todos os ensaios úteis começam com o carro parado e aceleram. Nesse tipo de ensaio, três coisas atrasam a resposta da mesma maneira: a zona morta DB (o carro demora a sair), a constante de tempo τ (a velocidade demora a chegar) e o atraso puro d . Uma pode fazer o papel da outra.

Para ver isso, fixamos d em vários valores e ajustamos os outros três parâmetros com os ensaios da bateria boa (Figura 1.10). O erro praticamente não muda (0,030 a 0,031 m/s), mas τ vai de 0,29 s a 0,01 s: o ajuste troca inércia por atraso puro sem piorar. Os dados não distinguem os dois. O ganho G , que define o patamar, é bem determinado; a *forma* do transitório, não.

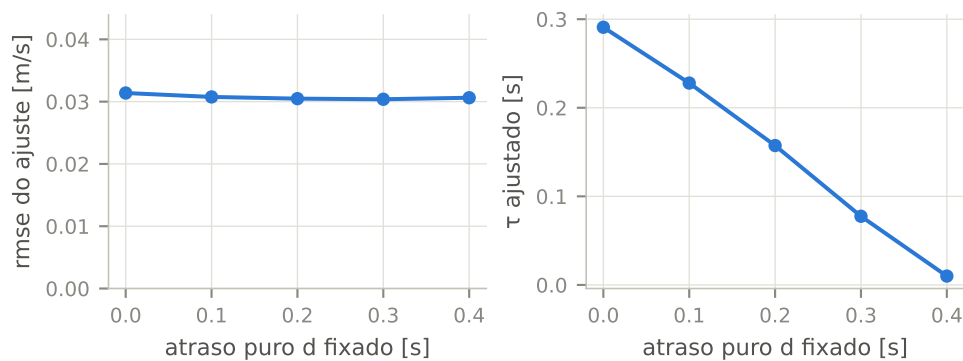


Figura 1.10: Perfil de custo: fixando o atraso puro d e ajustando o resto. O erro fica plano enquanto τ despenca — sinal de parâmetros não identificáveis com estes ensaios.

Ideia-chave

Um parâmetro só é identificável se o ensaio o excita de um jeito que os outros parâmetros não conseguem imitar. Para separar τ de d , seria preciso um ensaio em que o PWM suba e desça várias vezes, em frequências diferentes. Enquanto isso não acontece, o projeto precisa ser robusto a essa incerteza.

1.8 Passo 7: perícia — o que estava rodando no carro?

Antes de concluir qualquer coisa sobre o controlador, é preciso saber *qual* controlador rodou. Nos ensaios, versões diferentes foram enviadas ao carro, e nem sempre o envio funcionou. Os logs,

porém, guardam tudo o que é preciso para descobrir: o v e o v_{ref} que o controlador recebeu e o u que ele respondeu.

O método: pegar a lei de controle candidata, alimentá-la com o v e o v_{ref} registrados, na mesma ordem da Figura 1.2, e comparar o u produzido com o u registrado. Se a lei for a mesma, a diferença é zero até o arredondamento. Durante a parada do ultrassom, o controlador não roda (quem manda é o `set_vel(0)`), então o estado dele fica congelado nesses trechos.

ensaio	rmse, lei completa	rmse, lei sem o FF da referência
002158 a 004516 (nove ensaios)	0,0000	0,039 a 0,101
004851	0,0507	0,0000

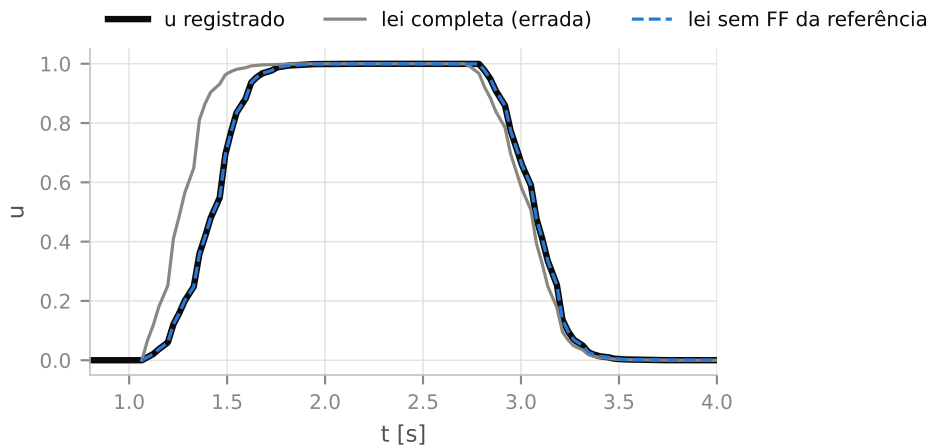


Figura 1.11: Ensaio 004851: só a lei sem o *feedforward* da referência reproduz o u registrado.

Resultado: até o 004516 rodou o controlador completo do simulador; no 004851, a versão sem o *feedforward* da referência. As outras variações testadas no código não chegaram a rodar no carro. A Figura 1.11 mostra a diferença: a lei certa se encaixa perfeitamente, a errada não.

1.9 Passo 8: um simulador do carro real

Testar cada ganho no carro é lento, gasta bateria e — como vimos — a bateria muda a planta. Com o modelo identificado, dá para montar um simulador do carro real e fazer o projeto nele. Para ser útil, ele precisa reproduzir o carro também *em malha fechada*, então inclui tudo o que afeta o laço:

- a planta (1.5) com os parâmetros da Tabela 1.2;
- a média de 30 amostras e a proteção do `set_u` ($|v| > 1,5 \text{ m/s} \Rightarrow u = 0$), mais a saturação $u \in [-1, 1]$;
- o período do laço sorteado da distribuição real (Figura 1.1) e o 1 s parado do `start_mission`;
- a ordem de operações da Figura 1.2.

A validação: rodar no simulador o controlador *antigo*, o mesmo que rodou no carro, e comparar com os ensaios (Figura 1.12).

ensaio	velocidade final [m/s]		tempo com $u = 1$ [s]	
	simulador	carro	simulador	carro
003330 (bateria boa, $v_{\text{ref}} = 1$)	1,88	1,85	1,02	0,99
003904 (descarregando, $v_{\text{ref}} = 1$)	1,74	1,75	1,44	1,49
004516 (mais fraca, $v_{\text{ref}} = 0,5$)	1,28	1,31	0,94	0,96

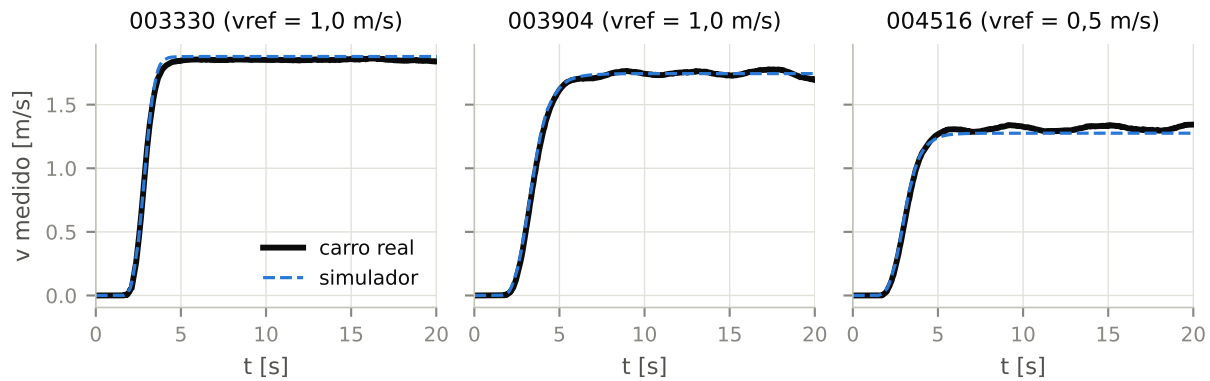


Figura 1.12: O simulador de malha fechada com o controlador antigo reproduz a catraca vista nos ensaios.

Armadilha

Esta validação não é cega: a planta foi ajustada nesses mesmos ensaios. O que ela prova é que a *montagem* da malha está certa — período do laço, média móvel, ordem das operações, saturações. A validação cega do simulador só vem com o primeiro ensaio do controlador novo no carro.

1.10 Passo 9: projete o controlador

1.10.1 A estrutura sai da planta

Cada termo do controlador do simulador existia por causa de algo na planta do simulador. A planta real é outra, então cada termo precisa ser revisto com a mesma pergunta: o que na planta justifica este termo?

no simulador	no carro real	por quê
PI	só P	A planta já tem um integrador (o PWM). Com só P, a malha é do tipo 1 e já tem erro zero em regime (ver abaixo). O I tornaria a malha do tipo 2: -90° a mais de fase em baixa frequência, mais sobressinal e <i>windup</i> , sem ganhar nada em regime.
compensação de atrito a_0, c	removida	Medido no Passo 6: com $u = 0$ a velocidade fica parada. O atrito já está dentro da curva PWM $\rightarrow v$. O termo vira um viés de $\approx 0,1$ em u (ver abaixo).
$u \in [0, 1]$	$u \in [-1, 1]$	$u < 0$ é o único jeito de baixar o acelerador. Com $u \geq 0$, a catraca do Passo 4.
filtro passa-baixas em u	removido	O integrador do PWM já filtra u ; o filtro só somava atraso.
FF da referência	mantido, dividido por K_{NOM}	Para seguir uma rampa de inclinação a , o integrador precisa de $u = a/K$: é a inversão do modelo.
erro = $v_{\text{des}} - v_{\text{med}}$	erro = $\text{MA30}(v_{\text{des}}) - v_{\text{med}}$	A medição chega 0,42s atrasada; a referência precisa do mesmo atraso (Seção 1.10.2).

Por que só P basta. Chame de $P_2(s)$ tudo o que vem depois do integrador do PWM (zona morta linearizada, ganho, inércia). Com um controlador proporcional K_P e uma perturbação constante δ somada *depois* do integrador — a bateria cedendo, uma subida, a zona morta —, o

erro é

$$E(s) = \frac{s}{s + k_\theta K_P P_2(s)} (R(s) - P_2(s) \Delta(s)). \quad (1.9)$$

O fator s no numerador vem do integrador do PWM. Pelo teorema do valor final, o erro em regime é zero tanto para uma referência em degrau quanto para uma perturbação em degrau. O integral do PI não acrescenta nada aqui.

Por que a compensação de atrito atrapalha. O raciocínio vale ao contrário para um viés b somado *antes* do integrador, direto em u . Em regime, o PWM só para se a entrada do integrador for zero, ou seja, $K_{Pe}/K_{NOM} + b = 0$:

$$e_{ss} = -\frac{b K_{NOM}}{K_P}. \quad (1.10)$$

A compensação de atrito é exatamente um viés desses, $b \approx 0,1$. Com os ganhos que vamos escolher, ela deixaria um erro de regime de cerca de 0,27 m/s. O simulador do Passo 8 confirma: somando $b = 0,1$ ao controlador escolhido, o erro de regime medido é de $-0,267$ m/s, o mesmo valor da fórmula. A compensação existia para cancelar um atrito que, no carro real, não aparece na entrada.

1.10.2 O atraso do sensor e a referência casada

A Figura 1.13 compara a fase da média de 30 amostras com o atraso de 0,05 s que o projeto do simulador assumia. A 1 rad/s, a média já consome 24°; a 4 rad/s, a banda do projeto antigo, consome 95°, contra 11° do atraso do simulador. Só isso já explica por que os ganhos do simulador não servem: com eles ($K_P = 4$, $K_I = 2,67$ e o filtro de 0,1 s em u), a malha real cruzaria 0 dB em 3,3 rad/s com margem de fase de -64° na bateria boa (-50° na fraca). Seria instável.

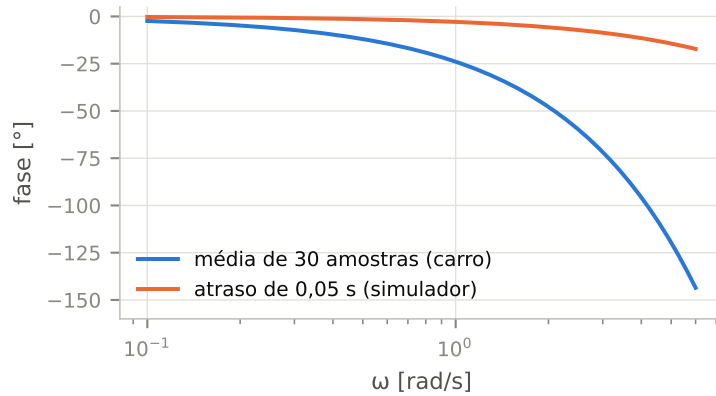


Figura 1.13: Fase introduzida pela média móvel de 30 amostras no carro (período de 28,8 ms) e pelo atraso de 0,05 s considerado no simulador. O eixo para em 6 rad/s porque em $2\pi/(NT) \approx 7,3$ rad/s a média tem seu primeiro zero: ela anula por completo um sinal nessa frequência.

O atraso também atrapalha o seguimento da rampa. Se o erro for $v_{des} - v_{med}$, durante uma rampa de $0,9 \text{ m/s}^2$ a velocidade medida fica $0,9 \times 0,42 \approx 0,38 \text{ m/s}$ abaixo da real, só por causa do sensor. O controlador “vê” esse erro, acelera a mais e passa da referência.

A solução é passar a referência pela *mesma* média móvel antes de compará-la com a medição (Figura 1.14). Os dois lados do erro ficam com o mesmo atraso, e o proporcional só reage ao erro real. Isso é um controlador de *dois graus de liberdade*: o caminho da referência (rampa, filtro, FF, média) é separado do caminho da realimentação (K_P). A referência fica fora da malha, então essa mudança não mexe na estabilidade; só melhora a resposta à referência.

A Figura 1.15 mostra o efeito, no simulador, com o mesmo K_P . O erro não some: a zona morta e a inércia continuam atrasando o carro em relação à referência, e isso é erro de verdade, que o

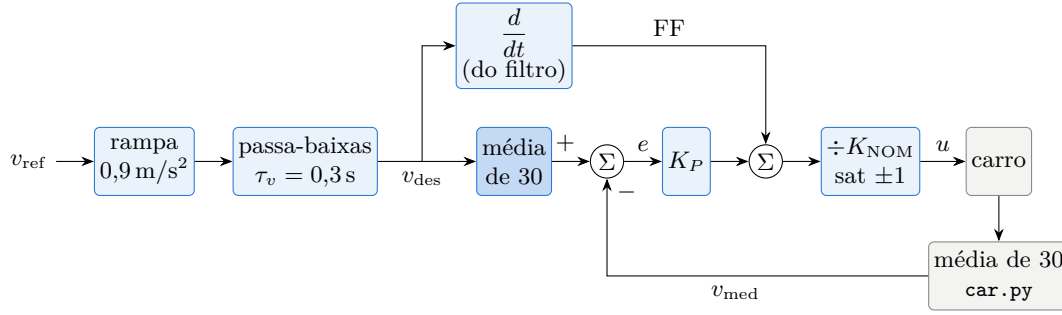


Figura 1.14: O controlador novo. A referência passa pela mesma média de 30 amostras que a medição (bloco destacado), e o *feedforward* vem da derivada da referência filtrada.

proporcional precisa corrigir. O casamento remove só a parte que era atraso do sensor. O pico do erro cai de 0,80 para 0,59 m/s e, mais importante, o erro quase não fica negativo depois da rampa ($-0,014$ contra $-0,079$ m/s). Esse trecho negativo é o controlador corrigindo um sobressinal que ele mesmo causou por ter acelerado a mais. Sem ele, o sobressinal cai de 8,2% para 1,5%.

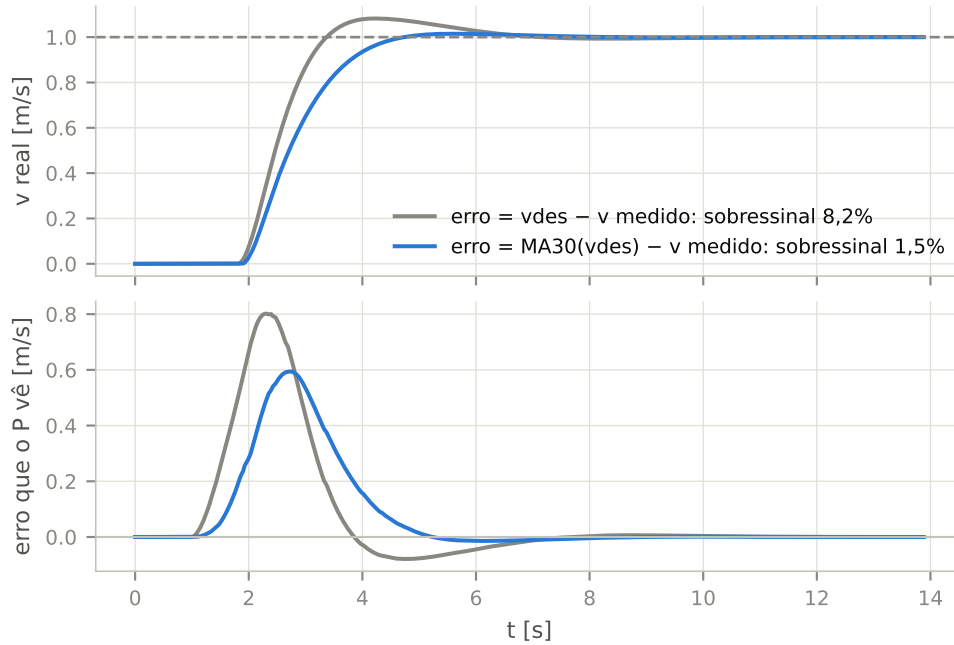


Figura 1.15: Com a referência casada (azul), o erro que o proporcional vê deixa de incluir o atraso do sensor: o pico cai e o trecho negativo depois da rampa quase some.

1.10.3 O ganho nominal K_{NOM}

A lei de controle é

$$u = \text{sat}_{\pm 1} \left(\frac{K_P (\text{MA30}(v_{\text{des}}) - v_{\text{med}}) + \dot{v}_{\text{des}}}{K_{\text{NOM}}} \right). \quad (1.11)$$

K_{NOM} é a estimativa de K que o controlador usa para converter aceleração desejada em u . O *feedforward* $\dot{v}_{\text{des}}/K_{\text{NOM}}$ só acerta a rampa se $K_{\text{NOM}} = K$.

Com a premissa de trabalho de que a bateria estará sempre carregada, a planta de projeto é a do grupo da bateria boa: $K = 1,49, 1,60$ e $1,61$ nos três ensaios. Daí $K_{\text{NOM}} = 1,6$. Se o K real for diferente, duas coisas acontecem:

- o *feedforward* erra a aceleração da rampa na mesma proporção, e o proporcional corrige o resto;

- o ganho de malha vira $K_P K/K_{\text{NOM}}$: com K maior, a malha fica mais rápida e com menos margem; com K menor, mais lenta e com mais margem.

Por isso o projeto é verificado com K de 1,3 a 2,0. O limite de cima não é exagero: a “bateria boa” dos ensaios não estava cheia — perdeu 34% de G em cerca de um minuto de uso —, então uma bateria cheia de verdade pode ter K acima de 1,6.

1.10.4 O ganho K_P

Com a lei (1.11) e a planta (1.5) linearizada acima da zona morta, a função de transferência de malha aberta é

$$L(s) = \frac{K_P}{K_{\text{NOM}}} K \frac{\text{MA30}(s) e^{-0,03s}}{s(\tau s + 1)}. \quad (1.12)$$

Três observações guiam a escolha:

- com $K = K_{\text{NOM}}$ e sem os atrasos, $L(s) = K_P/s$: um integrador cujo ganho cruza 0 dB em $\omega = K_P$. Por isso K_P tem unidade de rad/s: *ele é a banda da malha*;
- a média móvel tem fase $-\omega \cdot 0,42$ s: quanto maior a banda, mais fase ela come no cruzamento;
- o termo $e^{-0,03s}$ é uma margem para a janela do encoder (cerca de 15 ms) e o período da thread do atuador (cerca de 15 ms). Os ajustes deram $d \approx 0$, absorvido em τ ; os 30 ms ficam por segurança.

A Figura 1.16 mostra $L(j\omega)$ para $K_P = 0,6$ no caso nominal, com e sem a média móvel. Sem ela, a margem seria confortável; com ela, a fase cai rápido acima de 1 rad/s.

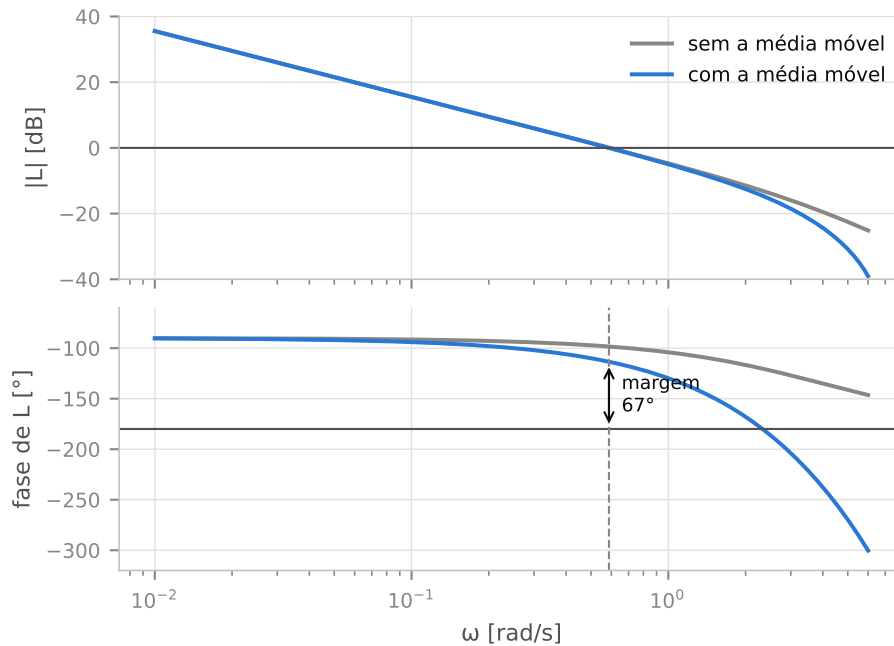
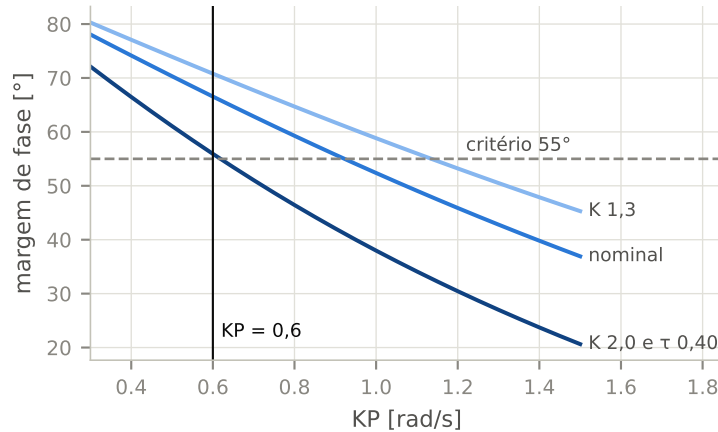


Figura 1.16: Diagrama de Bode da malha aberta com $K_P = 0,6$ (caso nominal: $K = 1,6$, $\tau = 0,25$ s). A média móvel é o que limita a margem.

Critério 1: margem de fase no pior caso plausível. A Tabela 1.3 e a Figura 1.17 dão a margem de fase para vários K_P e plantas. O pior caso é K alto com τ alto: K alto leva o cruzamento para frequências onde a fase da média cai rápido, e τ alto soma atraso na mesma frequência.

Tabela 1.3: Margem de fase [graus] da malha (1.12), com $K_{\text{NOM}} = 1,6$.

K_P	nominal (K 1,6, τ 0,25)	K 2,0	K 1,3	τ 0,40	K 2,0 e τ 0,40
0,4	74,2	70,3	77,1	71,0	66,5
0,6	66,6	61,1	70,8	62,1	55,9
0,8	59,3	52,4	64,7	53,9	46,4
1,0	52,4	44,3	58,8	46,4	38,0
1,2	45,9	36,9	53,2	39,6	30,5

**Figura 1.17:** Margem de fase em função de K_P para três plantas. O critério de 55° no pior caso limita K_P a 0,6.

Critério 2: resposta no tempo, no simulador não linear. A margem de fase é uma análise linear; o carro tem zona morta, saturação do PWM, saturação de u , período variável e desaceleração mais lenta que a aceleração (Seção 1.7.2). Todos esses efeitos estão no simulador do Passo 8. Para cada K_P , a Tabela 1.4 mostra o pior resultado entre $v_{\text{ref}} = 1,0$ e $0,5$ e o tempo para voltar a menos de 5% da referência depois de uma queda súbita de 20% no ganho da planta (bateria cedendo ou início de uma subida).

Tabela 1.4: Resposta no tempo no simulador do carro real (desaceleração 2 vezes mais lenta que a aceleração). A acomodação é contada desde o início da missão e inclui cerca de 1 s parado no `start_mission`.

K_P	sobressinal			acomodação	volta após
	nominal	K 2,0	pior caso	nominal	queda de 20%
0,6	1,5%	7,9%	14,0%	5,0 s	2,7 a 3,8 s
0,8	7,6%	16,4%	23,5%	5,9 s	2,3 a 3,0 s
1,0	15,4%	26,0%	33,8%	5,7 s	2,0 a 2,6 s
0,6 (sem casar)	8,2%	22,7%	27,7%	5,4 s	2,7 a 3,8 s

A escolha. O critério foi: o maior K_P com sobressinal nominal de no máximo cerca de 2% e margem de fase de pelo menos 55° no pior caso. O resultado é $K_P = 0,6$ rad/s. Subir K_P recupera perturbações cerca de 1 s mais rápido, mas o sobressinal cresce depressa (Figura 1.18). Descer deixa a recuperação mais lenta.

1.10.5 Robustez

A Figura 1.19 mostra o controlador escolhido com três valores de K . Com K menor (bateria cedendo), a resposta fica mais lenta e sem sobressinal; com K maior (bateria mais cheia que a

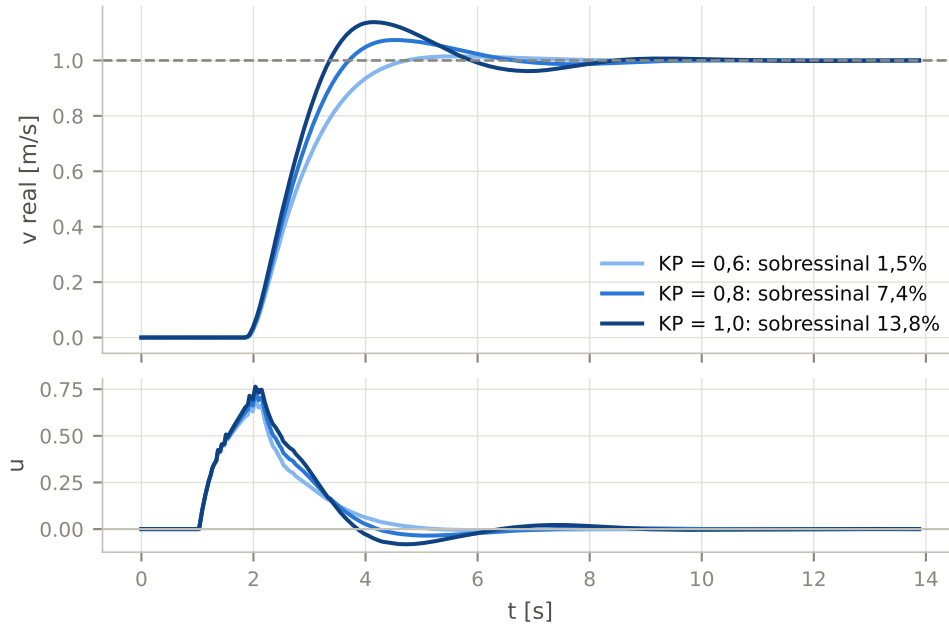


Figura 1.18: Resposta à rampa de referência até 1 m/s no caso nominal, com a desaceleração 2 vezes mais lenta que a aceleração. Acima de $K_P = 0,6$, o sobressinal cresce rápido, porque o atraso da média domina. (A figura mostra $v_{\text{ref}} = 1$; a Tabela 1.4 traz o pior caso entre $v_{\text{ref}} = 1$ e 0,5, por isso seus números são um pouco maiores.)

dos ensaios), aparece algum sobressinal, sem oscilar. Com as plantas da bateria que descarregava (ajustes próprios dos ensaios 003904 e 004516, com $K = 1,06$ e $0,88$), o controlador ainda acomoda, só mais devagar: sobressinal de 0,2 a 4% e acomodação em 5,5 a 6,6 s.

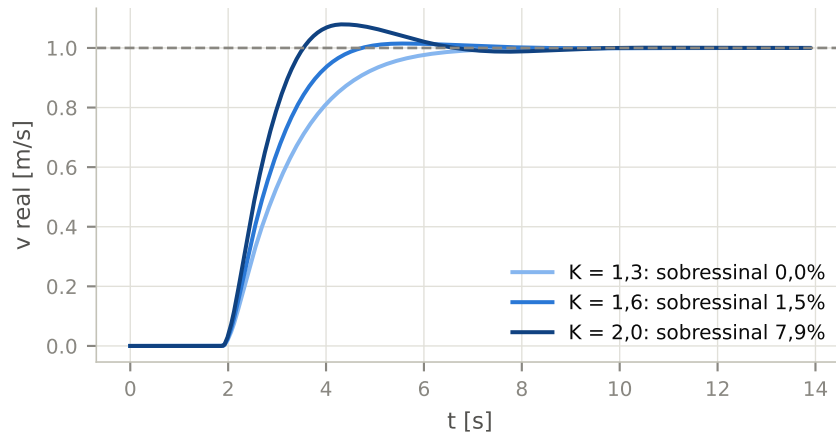


Figura 1.19: Controlador final ($K_P = 0,6$, $K_{\text{NOM}} = 1,6$) com $K = 1,3$, $1,6$ e $2,0$.

1.10.6 Verificar a implementação

O projeto foi feito com uma versão do controlador escrita para o simulador. O que vai para o carro é o código em `veiculo_real/main.py`. Para garantir que os dois são a mesma coisa, o `figuras.py` carrega o `main.py` de verdade (trocando as bibliotecas de hardware por módulos falsos), roda o `control_func` dele dentro do simulador e compara com a versão de projeto: a diferença máxima em u é da ordem de 10^{-16} , arredondamento de ponto flutuante.

veiculo_real/main.py (trecho do controlador)

```
def controlador_longitudinal(car, vdes_bruto):
    if car.dt > 0:
        _vdes_filt.alpha = np.clip(car.dt / TAU_VDES, 0.0, 1.0)
        vdes_anterior = _vdes_filt.value
        vdes = _vdes_filt.filter(vdes_bruto)
        feedforward_dvdes = (vdes_bruto - vdes_anterior) / TAU_VDES

        vdes_ma = _estado.setdefault('vdes_ma', MovingAverage(n=car.v_filt.n))
        erro = vdes_ma.filter(vdes) - car.v

        u = (KP_MOD * erro + feedforward_dvdes) / K_NOM
    return float(np.clip(u, -1.0, 1.0))
```

A Figura 1.20 fecha o capítulo: o mesmo carro simulado com o controlador antigo e com o novo.

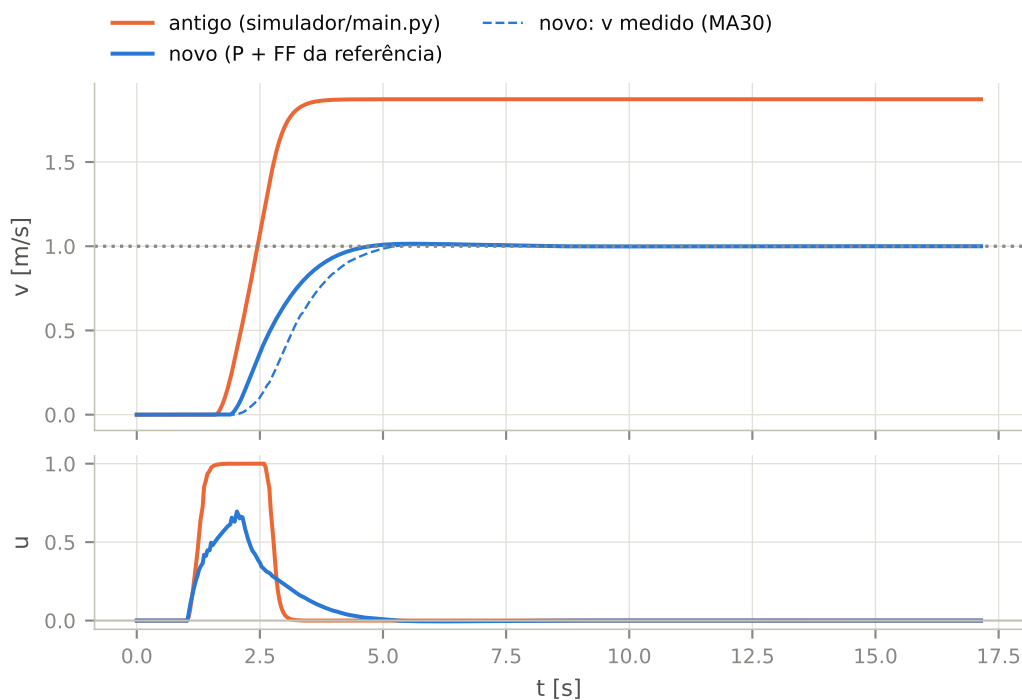


Figura 1.20: Controlador antigo e novo no simulador do carro real (bateria carregada). O antigo trava o acelerador; o novo chega a 1 m/s com cerca de 1,5% de sobressinal e usa $u < 0$ para devolver o acelerador.

1.11 O que ainda é hipótese

Um projeto honesto termina listando o que ainda não foi provado:

- **a bateria de projeto não estava cheia.** Uma bateria cheia pode ter K acima de 1,6. A robustez cobre até 2,0;
- **τ e d não foram separados** (Figura 1.10). O projeto considera τ até 0,4 s e 30 ms de atraso extra;
- **o simulador não foi validado às cegas.** O primeiro ensaio do controlador novo no carro cumpre esse papel. Com bateria carregada e $v_{\text{ref}} = 1$ m/s, a previsão é que a velocidade chegue a 1 m/s cerca de 3 s depois do início da rampa, sem passar de 1,02 m/s. Se passar muito disso, aquela bateria tem outro K ou outro τ — e o log desse ensaio é exatamente o dado para reidentificar;

- **o atraso da média de 30 amostras é o maior limitante.** Pela variação da saída da média em regime, o ruído do encoder bruto é da ordem de 0,03 m/s. Se isso se confirmar registrando o encoder sem filtro, uma média bem mais curta cortaria a maior parte do atraso e permitiria um K_P bem maior.

1.12 Resumo do método

O método em nove passos

1. **Código antes de dados.** Siga o comando até o atuador e a medição até o controlador. Tudo o que o código determina não é parâmetro: é estrutura.
2. **Modele só o resto**, com poucos parâmetros e significado físico.
3. **Faça inventário** (cópias, período real do laço) e **triagem com evidência** (IMU, comando, referência).
4. **Reconstrua** os sinais internos que o código permite calcular.
5. **Ajuste por erro de saída:** o modelo recebe só a entrada e prevê a trajetória inteira.
6. **Valide:** um teste físico (o que acontece com $u = 0$?), um ensaio com excitação diferente, previsão cega sem misturar condições e um perfil de custo para ver o que não é identificável.
7. **Descubra o que rodou**, reproduzindo o comando registrado.
8. **Monte um simulador de malha fechada** e valide-o contra os ensaios.
9. **Projete a partir da planta:** cada termo do controlador precisa de uma razão na planta; os ganhos saem de um critério na frequência (margem no pior caso) e de um no tempo (sobressinal, acomodação, recuperação de perturbação).

1.13 Exercícios

1. Calcule o atraso de grupo de uma média móvel de 10 amostras a 28,8 ms. Refaça a Tabela 1.3 com essa média no lugar da MA30 (use o `figuras.py`). Qual o maior K_P com margem de 55° no pior caso?
2. Mostre que, com um controlador PI em vez de P, a malha (1.12) passa a ter dois integradores. Desenhe o diagrama de Bode de fase em baixa frequência e explique por que isso reduz a margem de fase para a mesma banda.
3. Com a lei (1.11) e a compensação de atrito somada ($b = 0,1$), use o simulador para medir o erro de regime. Compare com $e_{ss} = -b K_{\text{NOM}}/K_P$.
4. Projete um ensaio que separe τ de d . Que sinal de u você usaria? Por quanto tempo? Como saberia, antes de rodar, que ele excita a diferença entre os dois?
5. No ensaio 003552, ajuste constantes de tempo separadas para subir e para descer. Use o resultado no simulador e verifique se a escolha de K_P muda.
6. A proteção do `set_u` faz $u = 0$ quando $|v| > 1,5$ m/s. Neste carro, o que essa proteção faz com o acelerador? Proponha uma alteração de uma linha que a torne efetiva e justifique com a equação (1.2).

Como reproduzir

Todos os números e figuras deste capítulo saem de um único script, o `figuras.py`, que lê os logs da pasta `veiculo_real/dados2_sem_feedforward/roxo`:

```
simulador/.venv/Scripts/python.exe veiculo_real/capitulo/figuras.py  
cd veiculo_real/capitulo  
pdflatex -aux-directory=build controle_carro_real.tex # duas vezes
```